

Machine Learning Aplicado à Previsão e Agrupamento de Anormalidade da Coluna Vertebral: Uma Análise Comparativa de Random Forest e K-Means

Diego Davi Kotz, Igor Rosa da Silva, Davi Gabriel Pereira Fiori,

João Pedro Heimovski Neves

Centro Universitário Assis Gurgacz - FAG Toledo

ddkotch@minha.fag.edu.br, irsilva6@minha.fag.edu.br,
dgpffiori@minha.fag.edu.br, jphneves@minha.fag.edu.br

ABSTRACT

This work addresses the use of supervised and unsupervised Machine Learning (ML) techniques to predict and group spinal column abnormality records. An initial dataset of 310 records was submitted to duplication checking and data imputation. For classification, the Random Forest algorithm achieved an accuracy of 81.72% on the test data, showing high performance (Recall of 0.82) in correctly identifying the majority class (abnormal records), but low precision for the minority class (normal records). The feature importance analysis revealed that the grade of spondylolisthesis is the most significant predictive factor (38.01%). In the unsupervised approach, the K-Means model with optimal $K=2$ resulted in an Adjusted Rand Index (ARI) of -0.0995, demonstrating a weak correlation between the actual abnormality status and the discovered clusters.

RESUMO

Este trabalho aborda o uso de técnicas de Aprendizagem de Máquina (AM), supervisionada e não supervisionada, com o objetivo de gerar previsões e agrupar registros de anormalidade na coluna vertebral. Um conjunto de dados inicial de 310 registros foi submetido à verificação de duplicidade e a substituição de dados. Para a classificação, o algoritmo Random Forest obteve uma acurácia de 81,72% nos dados de teste, mas apresentou alta performance (Recall de 0.82) na identificação correta da classe majoritária (registros anormais), mais apresentou baixa precisão na classe minoritária (registros normais). A análise de importância revelou que o grade of spondylolisthesis é o fator preditivo mais significativo (38,01%). Na abordagem não supervisionada, o modelo K-Means, com $K=2$ ideal, resultou em um Índice Rand Ajustado (ARI) de -0.0995, demonstrando fraca correlação entre o status real da anormalidade e os clusters descobertos.

1. INTRODUÇÃO

A anormalidade na coluna vertebral representa um desafio significativo na saúde pública global. A aplicação de técnicas de Aprendizagem de Máquina tem se mostrado promissora na identificação de padrões e fatores de risco, auxiliando no diagnóstico precoce.

O presente trabalho visa avaliar o desempenho do algoritmo de classificação Random Forest na previsão do status de anormalidades na coluna vertebral e, em paralelo, explorar se o algoritmo não supervisionado K-Means é capaz de identificar grupos de registros com base em seus perfis de risco. Para isso, foi utilizada uma base de dados abrangente de registros, retirados do site kaggle.

2. FUNDAMENTAÇÃO TEÓRICA

2.1. Random Forest

O Random Forest é um algoritmo de aprendizado de máquina supervisionado, amplamente adotado e registrado por Leo Breiman e Adele Cutler, conhecido por sua robustez e flexibilidade. O algoritmo opera combinando as saídas de múltiplas Árvores de Decisão (Decision Trees) para chegar a um resultado único e mais preciso. Sua capacidade de lidar eficientemente com problemas tanto de classificação quanto de regressão impulsionou sua popularidade na área de ciência de dados.

2.1.1. Árvores de Decisão

A base estrutural do Random Forest é a Árvore de Decisão. Este algoritmo de aprendizado supervisionado começa com uma questão básica e prossegue com uma série de perguntas subsequentes que formam os nós de decisão. Estes nós atuam como divisores de dados, direcionando as observações que atendem a critérios específicos para um ramo e as que não atendem para um caminho alternativo, culminando em uma decisão final, denominada nó folha.

As árvores de decisão são tipicamente treinadas pelo algoritmo de Árvore de Classificação e Regressão (CART). A qualidade dessas divisões é avaliada por métricas específicas, como a Impureza de Gini, o Ganho de Informação ou o Erro Quadrático Médio (MSE).

Apesar de sua utilidade, árvores de decisão individuais estão sujeitas a problemas como viés e, principalmente, overfitting (sobreajuste), onde o modelo se ajusta excessivamente aos dados de treinamento. O algoritmo Random Forest surge para mitigar esses desafios, agregando a previsão de múltiplas árvores pouco correlacionadas, resultando em previsões mais precisas.

2.1.2. Métodos de Conjunto

O algoritmo Random Forest se enquadra na categoria de Métodos de Aprendizado de Conjunto. Estes métodos utilizam um conjunto de classificadores (como as Árvores de Decisão) e agregam suas previsões individuais para identificar o resultado mais popular ou a média mais precisa.

O Random Forest é, especificamente, uma extensão do método Bagging (também conhecido como Bootstrap Aggregating), introduzido por Leo Breiman em 1996.

2.1.3. Bagging

No Bagging, amostras aleatórias de dados são selecionadas com substituição (amostras bootstrap) do conjunto de treinamento. Vários modelos são, então, treinados independentemente sobre essas amostras. Dependendo da tarefa (regressão ou classificação), a média ou a maioria dos votos dessas previsões agregadas resulta em uma estimativa mais robusta. O Bagging é uma técnica fundamental para reduzir a variância em um conjunto de dados com ruído.

2.1.4. Algoritmo de Random Forest em Detalhe

O Random Forest aprimora o Bagging ao introduzir a Aleatoriedade de Características (Feature Bagging). Essa é a diferença fundamental em relação às árvores de decisão comuns:

1. Bagging: Cada árvore é treinada em uma amostra bootstrap aleatória dos dados.
2. Aleatoriedade de Características: Em cada nó de cada árvore, o algoritmo não considera todas as possíveis divisões de características, mas apenas um subconjunto aleatório delas.

Esta dupla aleatoriedade (nos dados e nas características) é crucial, pois garante uma baixa correlação entre as árvores de decisão. Ao considerar esta variabilidade potencial, o Random Forest reduz significativamente o risco de overfitting, o viés e a variância geral, fornecendo previsões mais acuradas.

2.1.4. Como Funciona a Previsão

O algoritmo é composto por uma coleção de Árvores de Decisão. Para a previsão, o modelo aplica às seguintes regras:

- Regressão: A previsão final é a média das saídas das árvores de decisão individuais.

- Classificação: A previsão final é determinada pela maioria dos votos (a classe categórica mais frequente) entre as árvores.

Além disso, uma parte da amostra de treinamento, não utilizada por cada árvore (out-of-bag - oob), é reservada para validação cruzada, finalizando o processo de previsão.

2.1.5. Benefícios e Aplicações

O Random Forest é valorizado por sua capacidade de reduzir o risco de overfitting e pela sua flexibilidade em lidar com ambos os tipos de tarefas de aprendizado. Além disso, o algoritmo facilita a avaliação da Importância das Características, o que pode ser medido por métodos como a Impureza de Gini ou a Importância por Permutação (MDA), auxiliando na compreensão da contribuição de cada variável para o modelo.

Graças a essas características, o Random Forest é aplicado em diversos setores, como:

- Setor Financeiro: Avaliação de risco de crédito, detecção de fraudes e precificação de opções.
- Setor de Saúde: Classificação de expressões gênicas e descoberta de biomarcadores.
- E-commerce: Desenvolvimento de mecanismos de recomendação.

2.2. Agrupamento K-means

O Agrupamento K-means é um dos algoritmos de aprendizado de máquina não supervisionado mais populares para agrupamento (clustering) de dados. Sua função principal é agrupar pontos de dados não rotulados em K grupos (clusters) com base na similaridade entre eles.

Ao contrário do aprendizado supervisionado, o K-means utiliza dados de treinamento sem rótulos definidos. Ele se classifica como um método de agrupamento exclusivo (hard), onde cada ponto de dado pertence estritamente a apenas um cluster. O K-means é um algoritmo de agrupamento baseado em centróides e opera de forma iterativa, buscando dividir um conjunto de dados em grupos semelhantes e minimizando a distância entre os pontos e o centróide (o centro do cluster, tipicamente a média) ao qual são atribuídos.

2.2.1 Funcionamento do Agrupamento K-means

O algoritmo K-means é um processo iterativo que busca minimizar a Soma dos Erros Quadrados (SSE), que é a soma das distâncias entre os pontos de dados e seus centros de agrupamento atribuídos.

O processo segue uma abordagem de duas etapas (Expectation-Maximization - EM):

1. Inicialização de K: O algoritmo começa definindo o número de clusters desejados (K) e inicializando os K centróides, geralmente por seleção aleatória.
2. Atribuição e Recalculo (Iteração):
 - Atribuição: Cada ponto de dado é atribuído ao seu centróide mais próximo, utilizando uma medida de distância, como a Euclidiana.
 - Recalculo: O centróide de cada cluster é recalculado, definindo-se a média de todos os pontos recém-atribuídos.

Este ciclo se repete até que os centróides atinjam a convergência (suas posições se estabilizam) ou até que o número máximo de iterações seja atingido.

2.2.2. Métricas de Avaliação de Clusters

A qualidade dos clusters é avaliada pela busca por alta coesão (similaridade entre pontos no mesmo cluster) e alta separação (distinção entre clusters).

- Inércia (Soma dos Erros Quadrados - SSE): É a métrica primária que o K-means busca minimizar. Calcula a soma das distâncias quadradas de cada ponto ao centróide do seu cluster. Um valor de inércia menor indica que os pontos dentro do cluster são mais compactos.
- Índice de Dunn: Avalia a separação dos clusters, representando a relação entre a distância mínima intercluster e a distância máxima intracluster. Um alto Índice de Dunn indica clusters bem separados.

2.2.3. Otimização do Desempenho do K-mean

O K-means não é um algoritmo determinístico devido à sua inicialização aleatória. Portanto, a otimização da seleção dos centróides iniciais e do número ideal de K é crucial.

- K-means++ (Inicialização): Para mitigar o risco de resultados incoerentes da inicialização puramente aleatória, o algoritmo K-means++ otimiza a seleção dos centros iniciais, escolhendo centróides subsequentes com base na probabilidade de estarem mais distantes dos centros já escolhidos, garantindo uma distribuição inicial mais dispersa.

- Método do Cotovelo (Elbow Method): É uma técnica gráfica para encontrar o número ideal de clusters (K). Envolve calcular o WCSS (Within-Cluster Sum of Squares)/Inércia para uma faixa de valores de K e plotar o resultado. O valor de K onde a diminuição do WCSS se nivela (formando o "cotovelo") é considerado o ideal, pois representa o ponto de equilíbrio onde ganhos adicionais são marginais.

2.2.4 Vantagens e Desafios do K-means

O K-means possui diversas características que impulsionam sua popularidade. Ele é simples de entender e aplicar, rápido devido à sua abordagem computacionalmente simples e escalável para grandes conjuntos de dados.

Entretanto, o K-means enfrenta desafios que requerem atenção:

- Dependência de Parâmetros de Entrada: O desempenho do agrupamento depende criticamente da inicialização correta dos centróides e da seleção do valor K adequado. Uma inicialização incorreta pode levar a um maior tempo de execução e clusters de baixa qualidade.
- Sensibilidade a Valores Discrepantes (Outliers): O K-means é muito sensível a outliers, pois o cálculo do centróide pela média é facilmente distorcido por valores extremos. Essa sensibilidade pode levar à inclusão desses valores no overfitting.
- Desempenho em Certos Conjuntos de Dados: O K-means pressupõe que os clusters tenham formas esféricas e tamanhos semelhantes, tendo um desempenho insatisfatório em conjuntos de dados com clusters de tamanhos desiguais, variações de densidade ou formatos irregulares.

2.2.4 Aplicações Comuns

O algoritmo K-means é aplicado em vários setores, sendo particularmente útil em tarefas como:

- Segmentação de Clientes: Dividir clientes em grupos com base em características comuns para direcionamento de campanhas.
- Segmentação de Imagem: Dividir imagens digitais em conjuntos distintos de pixels.
- Mecanismos de Recomendação: Utilização em combinação com outras técnicas para gerar sugestões de produtos em plataformas de e-commerce.
- Classificação de Documentos: Alocar documentos em categorias específicas para moderação ou organização.

2.3. Fatores de Anormalidades na Coluna Vertebral

2.3.1. Relevância e Definição das Anormalidades na Coluna Vertebral

As anormalidades da coluna vertebral são as principais causas de dores crônicas e incapacidade funcional, assim representando um problema de saúde pública global. Segundo a OMS (Organização Mundial de Saúde) apontam que condições musculoesqueléticas, com predominância as que afetam a coluna, como maior responsável por causar incapacidade no mundo, afetando milhões de pessoas de todas as faixas etárias.

As anormalidades na coluna englobam várias condições que comprometem a estrutura, alinhamento e a função da coluna, afetando a medula espinhal, raízes nervosas e elementos de suporte (vértebras, discos e ligamentos). Sendo as anormalidades mais comuns Hérnia de Disco, Estenose Espinhal, Escoliose e Cifose e Doença Discal Degenerativa.

2.3.2. Fatores de Risco para Anomalias na Coluna Vertebral: Classificação

Fatores de risco são características ou condições que aumentam a probabilidade de um indivíduo desenvolver uma anormalidade na coluna vertebral. Eles são classicamente divididos em várias categorias, dependendo da possibilidade de intervenção.

2.3.2.1. Fatores de Risco Não Modificáveis

São aqueles que não podem ser alterados pelo indivíduo, mas que devem ser monitorados para uma gestão de risco mais rigorosa:

- Envelhecimento: O desgaste natural das articulações e a redução muscular com o passar da idade podem apresentar alterações posturais progressivamente.
- Genética: Algumas alterações posturais podem ter fatores hereditários, assim aumentando o aumento da predisposição ao problema.
- Doenças ósseas e neuromusculares: Condições como osteoporose, artrite e distrofias musculares podem ser fatores que influenciam a estabilidade da coluna e na postura com o passar do tempo.
- Sexo: Mulheres em período menstrual, gravidez, endometriose e menopausa possuem mais condições para anormalidades na coluna vertebral.

2.3.2.2. Fatores de Risco Modificáveis e Variáveis Bioquímicas

Estes fatores podem ser alterados ou controlados através de mudanças no estilo de vida ou tratamento médico. Eles representam o foco primário das estratégias de prevenção:

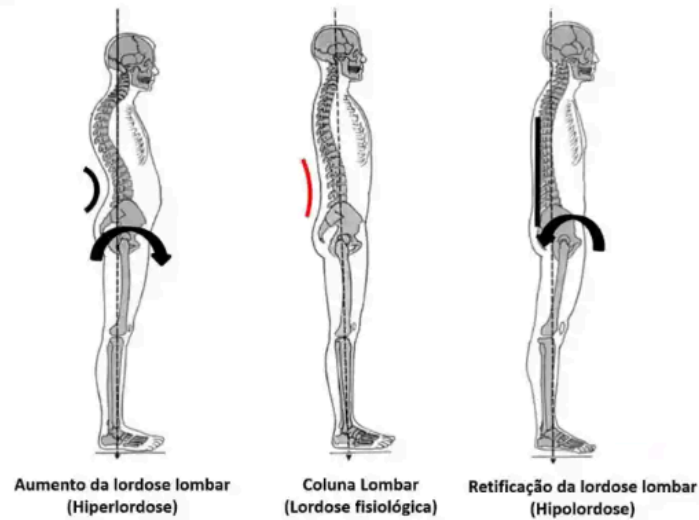
- Postura inadequada: Comprometimento do alinhamento da coluna e geração de sobrecarga muscular devido sentar-se e caminhar de forma incorreta.
- Sedentarismo: Enfraquecimento da musculatura que sustenta a coluna aumenta o risco de desalinhamento da postura sendo causado pela falta de atividade física.
- Uso excessivo de dispositivos eletrônicos: Má postura devido hábito de inclinar a cabeça para baixo na utilização de celulares e tablets sobrecarregam a coluna cervical.
- Transporte inadequado de peso e carga ocupacional elevada: Desequilíbrios posturais e dores na coluna são resultados de transporte inadequado de peso, sendo no dia a dia ou no trabalho, sendo um fator que influencia todas as faixas etárias.
- Diabetes mellitus: Um fator de risco potencial para alterações na coluna que amplifica mecanismos para o seu desenvolvimento, sendo por inflamação crônica e redução da irrigação das placas vertebrais e glicação de proteínas tornando mais frágil e rígida.
- Tabagismo: Estudo considera que apenas o tabagismo não é um fator predominante para anormalidades na coluna, mas sim uma variável de combinação com outras variáveis, como sobrepeso, carga ocupacional elevada, assim associado a lombalgia.
- Obesidade: Estudos indicaram alterações nas placas terminais e infiltrações gordurosas nos músculos paravertebrais em todos os níveis lombares.

2.3.3. Variáveis na Biomecânica da Coluna Lombar e Pelve

O dataset é subdividido em variáveis que determinam condições de anormalidades da coluna vertebral, assim feitas através de exames radiológicos, sendo abaixo apresentada:

- Lordose Lombar (LL): Refere-se à curvatura anterior da coluna lombar, localizada no segmento inferior do tronco, sendo em casos de anormalidade da curvatura descritas como hiperlordose se há aumento e hiperlordose ou retificação lombar na diminuição.

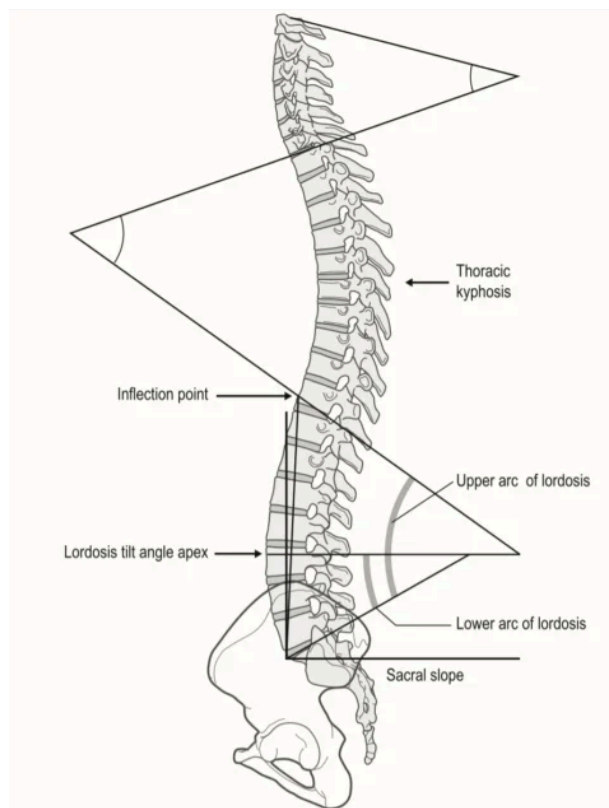
Figura 1 – Ilustração da Lordose Lombar



Fonte: Captura de tela do site Dr. Ricardo Teixeira (s.d.).

- Inclinação Sacral (IS): É um parâmetro no equilíbrio espinopélvico, indica o ângulo entre o platô superior de S1 e uma linha horizontal.

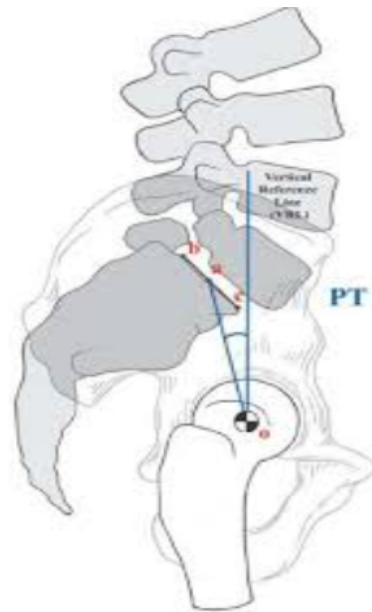
Figura 2 – Ilustração da Inclinação Sacral



Fonte: Captura de tela do site radiologia online (s.d.).

- Versão Pélvica (VP): É um parâmetro que mede o ângulo entre uma linha que liga o ponto médio do platô sacral até o eixo da cabeça femoral, e uma linha vertical perpendicular ao solo.

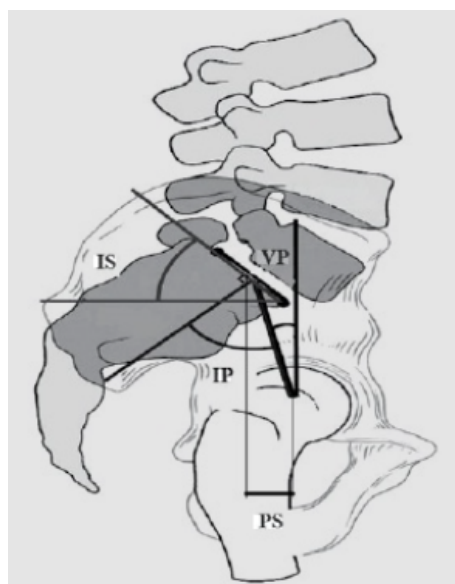
Figura 3 – Ilustração da Versão Pélvica



Fonte: Captura de tela do site researchgate (s.d.).

- Incidência Pélvica (IP): É uma técnica definida através da fórmula da soma da inclinação sacral e versão pélvica, definida como ângulo entre uma linha ao ponto médio do platô sacral e a uma linha até o eixo central da cabeça femoral. Sendo assim a fórmula $IP = IS + VP$.

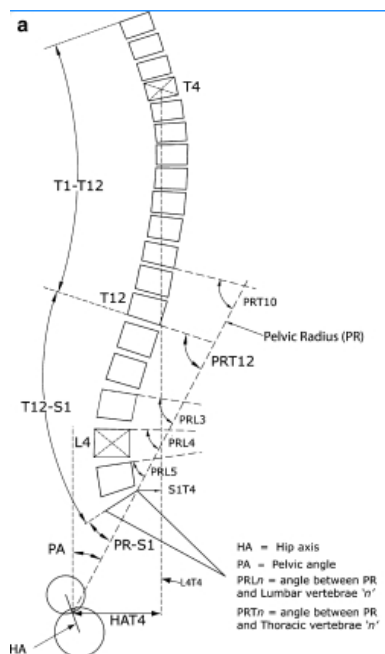
Figura 4 – Ilustração da Incidência Pélvica



Fonte: Captura de tela do site scielo br(s.d.).

- Raio Pélvico (RP): É uma técnica que traça uma linha reta, entre o eixo do quadril e o canto posterior da placa terminal S1, assim o ângulo formado entre a linha RP e a placa terminal sacral, o RP1, sendo a medida do desenvolvimento da morfologia sacropélvica.

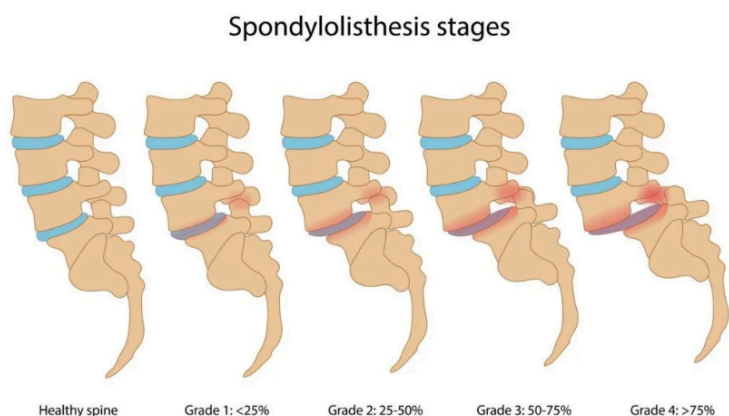
Figura 5 – Ilustração da Raio Pélvico



Fonte: Captura de tela do site pubmed (s.d.).

- Grau de Espondilolistese: A espondilolistese, conhecida também por anterolistese, é o deslizamento vertical da coluna, sendo definido como deslizamento ou escorregamento entre vértebras, causando saliências na coluna. Sendo através da classificação Meyerding, descrevendo os graus deste deslizamento.

Figura 6 – Ilustração do Grau de Espondilolistese



Fonte: Captura de tela do site neurocirurgiasp (s.d.).

3. Metodologia

A metodologia aplicada neste estudo seguiu as etapas de aquisição, pré-processamento, engenharia de recursos, análise exploratória e modelagem de Machine Learning.

3.1. Aquisição e Limpeza de Dados

O conjunto de dados (Vertebral Column Data Set) foi obtido no repositório Kaggle no formato CSV, contendo 310 registros iniciais. Para garantir a reprodutibilidade no ambiente de execução (Google Colab), o arquivo foi armazenado e acessado via Google Drive. A lista completa de bibliotecas utilizadas no projeto está detalhada no Trecho de Código 1.

Trecho de Código 1

```
from google.colab import drive
import pandas as pd
import matplotlib.pyplot as plt
from IPython.display import display, Markdown, HTML
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    roc_auc_score,
    roc_curve
)
import numpy as np
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import adjusted_rand_score,
normalized_mutual_info_score
```

A extração dos dados iniciou-se com a montagem do repositório Google Drive, utilizando a biblioteca google.colab, conforme o Trecho de Código 2.

Trecho de Código 2

```
drive.mount('/Colab_Notebooks')
```

Em seguida, o arquivo CSV foi lido e convertido para o objeto DataFrame (dados) utilizando a biblioteca pandas, conforme o Trecho de Código 3.

Trecho de Código 3

```
caminho = '/Colab_Notebooks/My Drive/Colab Notebooks/Trabalho Final  
Inteligência Artificial/DatasetColunaVertebral.csv'  
df = pd.read_csv(caminho)
```

Na fase de verificação, foram verificados se o dataset possui valores duplicados e nulos, assim apresentados no Trecho de Código 4.

Trecho de Código 4

```
num_registros = len(df)  
print(f"Número de registros: {num_registros}")  
total_duplicatas = df.duplicated().sum()  
print(f"Total de linhas duplicadas: {total_duplicatas}")  
print("\nContagem de dados nulos por coluna:")  
print(df.isnull().sum())
```

3.2. Engenharia de Features

Para adequar o dataset aos modelos de Machine Learning, realizou-se a Engenharia de Recursos. Esta etapa focou na codificação das features, eliminando o excesso de números após a vírgula e a mudança dos dados da variável *Class_att* de Abnormal para Anormal. O processo de codificação para todas as 6 features utilizadas no modelo está detalhado no Trecho de Código 5.

Trecho de Código 5

```
# Eliminado o excesso de casas após a virgula  
df.iloc[:, :6] = df.iloc[:, :6].round(2)  
# Substituição do Abnormal por Anormal  
df['Class_att'] = df['Class_att'].replace('Abnormal', 'Anormal')
```

3.3. Análise Exploratória

Após as etapas de verificação e transformação (Trecho de Código 4 e 5), foi gerado o código para uma análise de acordo com dados levantados sobre as métricas consideráveis normal e anormal, de acordo com o artigo *Incidência Pélvica: um parâmetro fundamental para definição do equilíbrio sagital da coluna vertebral e Classificação de Meyerding*, demonstrado no trecho de código 6, trazendo valores de porcentagem de cada variável.

Trecho de Código 6

```
print('ANALISE EXPLORATORIO DO CONJUNTO COLUNA VERTEBRAL')  
  
print('Valores exploratorios segundo Artigo "Incidência Pélvica: um  
parâmetro fundamental para definição do equilíbrio sagital da coluna  
vertebral" e "Classificação de Meyerding"')  
  
print(f"Total de registros: {len(df_tratados)} pacientes")
```

```

total = len(df_tratados)

classificacaoAnormal = round((len(df_tratados[df_tratados['Class_att']
== 'Anormal'])) / total)* 100, 2)

classificacaoNormal = round((len(df_tratados[df_tratados['Class_att'] ==
'Normal'])) / total)* 100, 2)

IP_Anormal = round((len(df_tratados[(df_tratados['pelvic incidence'] <
40) | (df_tratados['pelvic incidence'] > 65)))/total)*100, 2)

IP_Normal = round((len(df_tratados[(df_tratados['pelvic incidence'] >=
40) & (df_tratados['pelvic incidence'] <= 65)))/total)*100, 2)

VP_Anormal = round((len(df_tratados[(df_tratados[' pelvic tilt'] < 10) |
(df_tratados[' pelvic tilt'] > 25)))/total)*100,2)

VP_Normal = round((len(df_tratados[(df_tratados[' pelvic tilt'] >= 10) &
(df_tratados[' pelvic tilt'] <= 25)))/total)*100, 2)

LL_Anormal = round((len(df_tratados[(df_tratados[' lumbar lordosis
angle'] < 40) | (df_tratados[' lumbar lordosis angle'] >
80)))/total)*100, 2)

LL_Normal = round((len(df_tratados[(df_tratados[' lumbar lordosis
angle'] >= 40) & (df_tratados[' lumbar lordosis angle'] <=
80)))/total)*100, 2)

IS_Anormal = round((len(df_tratados[(df_tratados[' sacral slope'] < 30)
| (df_tratados[' sacral slope'] > 50)))/total)*100, 2)

IS_Normal = round((len(df_tratados[(df_tratados[' sacral slope'] >= 30)
& (df_tratados[' sacral slope'] <= 50)))/total)*100, 2)

RP_Anormal = round((len(df_tratados[(df_tratados[' pelvic radius'] <
83.7)))/total)*100, 2)

RP_Indefinido = round((len(df_tratados[(df_tratados[' pelvic radius'] >
83.7) & (df_tratados[' pelvic radius'] < 92.9)))/total)*100, 2)

RP_Normal = round((len(df_tratados[(df_tratados[' pelvic radius'] >=
92.9)))/total)*100, 2)

# Função para classificar o grau de Meyerding
def classificar_meyerding(percentual):

    if percentual <= 0: return 'Grau 0 (normal)'

    elif percentual <= 25: return 'Grau I (1-25%)'

    elif percentual <= 50: return 'Grau II (26-50%)'

    elif percentual <= 75: return 'Grau III (51-75%)'

    elif percentual <= 100: return 'Grau IV (76-100%)'

    else: return 'Grau V (>100%)'

# Aplicar a classificação

```

```

df['Meyering'] = df_tratados[' grade of
spondylolisthesis'].apply(classificar_meyering)

# Calcular porcentagens

total = len(df_tratados)

contagem = df['Meyering'].value_counts()

porcentagens = (contagem / total) * 100

# Exibir resultados

print("Grau de Espondilolistese:")

ordem_graus = ['Grau 0 (normal)', 'Grau I (1-25%)', 'Grau II (26-50%)',
               'Grau III (51-75%)', 'Grau IV (76-100%)', 'Grau V
(>100%)']

dados_graus = {
    "Grau": [],
    "Registros": [],
    "Percentual (%)": []}

for grau in ordem_graus:
    qtd = contagem.get(grau, 0)
    pct = (qtd / total) * 100
    dados_graus["Grau"].append(grau)
    dados_graus["Registros"].append(qtd)
    dados_graus["Percentual (%)"].append(round(pct, 2))
    print(f"{grau}: {qtd} paciente(s) ({pct:.1f}%)")

# Normalização de dados

def is_anormal_pelvic_incidence(x): return (x < 40) | (x > 65)
def is_anormal_pelvic_tilt(x): return (x < 10) | (x > 25)
def is_anormal_lumbar_lordosis(x): return (x < 40) | (x > 60)
def is_anormal_sacral_slope(x): return (x < 30) | (x > 50)
def is_anormal_pelvic_radius(x): return x < 83.7
def is_anormal_grade(x): return x > 0

criterios = {
    'pelvic incidence': is_anormal_pelvic_incidence,
    ' pelvic tilt': is_anormal_pelvic_tilt,

```

```

    ' lumbar lordosis angle': is_anormal_lumbar_lordosis,

    ' sacral slope': is_anormal_sacral_slope,

    ' pelvic radius': is_anormal_pelvic_radius,

    ' grade of spondylolisthesis': is_anormal_grade

}

df_normalizado = pd.DataFrame(index=df_tratados.index)

for col, func in criterios.items():

    anormal = func(df[col])

    df_normalizado[col] = anormal.astype(int)

colunas_criterios = list(criterios.keys())

df_normalizado['total_anormalidade'] =
df_normalizado[colunas_criterios].sum(axis=1)

freq_abs =
df_normalizado['total_anormalidade'].value_counts().sort_index()

freq_rel = freq_abs / len(df_normalizado) * 100

resumo = pd.DataFrame({

    'Nº de anormalidades': freq_abs.index,

    'Frequência (registros)': freq_abs.values,

    'Porcentagem (%)': freq_rel.values})

resumo['Porcentagem (%)'] = resumo['Porcentagem (%)'].round(2)

```

Assim através da biblioteca *from IPython.display import display, Markdown, HTML*, demonstrando assim a classificação das variáveis de acordo com os dados dos artigos citados acima, sendo demonstrado no trecho de código 6 e visualmente através de tabelas apresentando os percentuais na imagem Figura 7.

Trecho de Código 7

```

# 1. Cabeçalho principal

display(Markdown("#  Análise Exploratória - Coluna Vertebral"))

display(Markdown("**Referências:** *Incidência Pélvica: um parâmetro fundamental para definição do equilíbrio sagital da coluna vertebral* e *Classificação de Meyerding*"))

display(Markdown(f"**Total de registros:** {len(df_tratados)} registros"))

# 2. Cards com métricas gerais

display(Markdown("##  Visão Geral"))

html_cards = f"""

```



```

<div style="display: flex; gap: 20px; margin-bottom: 20px; justify-content: center;">

    <div style="background-color: #f8f9fa; padding: 15px; border-radius: 10px; box-shadow: 2px 2px 5px rgba(0,0,0,0.1); text-align: center; flex:1; max-width: 300px;">

        <h3>✅ Normais</h3>

        <p style="font-size: 32px; color: green;">{(classificacaoNormal)::2f}%</p>

        <p>{(int(len(df_tratados)*((classificacaoNormal)/100)))registros}</p>

    </div>

    <div style="background-color: #f8f9fa; padding: 15px; border-radius: 10px; box-shadow: 2px 2px 5px rgba(0,0,0,0.1); text-align: center; flex:1; max-width: 300px;">

        <h3>⚠️ Anormais</h3>

        <p style="font-size: 32px; color: red;">{(classificacaoAnormal)::2f}%</p>

        <p>{(int(len(df_tratados)*((classificacaoAnormal)/100)))registros}</p>

    </div>

</div>

"""

display(HTML(html_cards))

# ESTILO GLOBAL PARA TODAS AS TABELAS

estilo_tabelas = '''

<style>

    .tabela-centralizada {

        display: flex;

        justify-content: center;

        margin: 20px 0;

        width: 100%;

    }

    .tabela-centralizada table {

        border-collapse: collapse;

        width: auto;

        min-width: 80%;

    }

'''

```

```

        box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
    }

    .tabela-centralizada th, .tabela-centralizada td {
        padding: 10px 15px;
        border: 1px solid #ddd;
        text-align: center;
    }

    .tabela-centralizada th {
        background-color: #f2f2f2;
        font-weight: bold;
    }

    .tabela-centralizada td:first-child {
        text-align: left;
    }
</style>
'''

display(HTML(estilo_tabelas))

# 3. Tabela de critérios individuais (com barra visual)
display(Markdown("## 🦴 Critérios Radiológicos"))

dados_criterios = {
    "Critério": ["Incidência Pélvica", "Versão Pélvica", "Lordose
Lombar", "Inclinação Sacral"],

    "Anormais (%)": [IP_Anormal, VP_Anormal, LL_Anormal, IS_Anormal],

    "Normais (%)": [IP_Normal, VP_Normal, LL_Normal, IS_Normal],

    "Faixa normal": ["40-65°", "10-25°", "40-80°", "30-50°"]}

df_crit = pd.DataFrame(dados_criterios)

def barra(pct, width=20):
    filled = int(width * pct / 100)

    bar = (f'<span style="color: #ff4444;">{"█" * filled}</span>'

          f'<span style="color: #4CAF50;">{"█" * (width -
filled)}</span>')

    return f"{bar} {pct:.1f}%"

```

```

df_crit["Distribuição"] = df_crit["Anormais (%)"].apply(barra)

df_crit["Anormais (%)"] = df_crit["Anormais (%)"].apply(lambda x:
f"{x:.2f}")

df_crit["Normais (%)"] = df_crit["Normais (%)"].apply(lambda x:
f"{x:.2f}")

html_tabela3 = df_crit.to_html(escape=False, index=False)

display(HTML(f'<div class="tabela-centralizada">{html_tabela3}</div>'))

dados_raio = {

    "Distribuição": ["Anormal", "Indefinido", "Normal"],

    "Faixas": ["< 83.7", "83.7 - 92.9", "92.9 >"],

    "Quantidade de Registros": [int(total*(RP_Anormal/100)),
int(total*(RP_Indefinido/100)), int(total*(RP_Normal/100))],

    "Percentual (%)": [RP_Anormal, RP_Indefinido, RP_Normal]}

df_crit = pd.DataFrame(dados_raio)

html_tabela4 = df_crit.to_html(escape=False, index=False)

display(HTML(f'<div class="tabela-centralizada">{html_tabela4}</div>'))

# 4. Grau de Espondilolistese (tabela colorida)

display(Markdown("## 🦴 Grau de Espondilolistese (Classificação de
Meyerdin)"))

df_graus = pd.DataFrame(dados_graus)

html_tabela5 = df_graus.to_html(index=False)

display(HTML(f'<div class="tabela-centralizada">{html_tabela5}</div>'))

# 5. Distribuição do número de anormalidades

display(Markdown("## 📊 Distribuição do Número de Anormalidades por
Registro"))

df_dist = pd.DataFrame({

    'Nº de anormalidades': resumo['Nº de anormalidades'],

    'Frequência (registros)': resumo['Frequência (registros)']})

df_dist["Distribuição"] = resumo["Porcentagem (%)"].apply(lambda x:
barra(x, width=50))

html_tabela6 = df_dist.to_html(escape=False, index=False)

display(HTML(f'<div class="tabela-centralizada">{html_tabela6}</div>'))

```

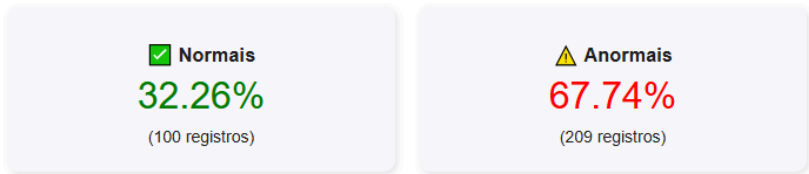
Figura 7 – Resultado da Análise Exploratória

Análise Exploratória - Coluna Vertebral

Referências: Incidência Pélvica: um parâmetro fundamental para definição do equilíbrio sagital da coluna vertebral e Classificação de Meyerding

Total de registros: 310 registros

Visão Geral



Critérios Radiológicos

Critério	Anormais (%)	Normais (%)	Faixa normal	Distribuição
Incidência Pélvica	49.35	50.65	40-65°	49.4%
Versão Pélvica	40.00	60.00	10-25°	40.0%
Lordose Lombar	37.10	62.90	40-80°	37.1%
Inclinação Sacral	45.16	54.84	30-50°	45.2%

Raio Pélvico

Distribuição	Faixas	Quantidade de Registros	Percentual (%)
Anormal	< 83.7	3	1.29
Indefinido	83.7 - 92.9	7	2.26
Normal	92.9 >	298	96.45

☐ Grau de Espondilolistese (Classificação de Meyerding)

Grau	Registros	Percentual (%)
Grau 0 (normal)	58	18.71
Grau I (1-25%)	116	37.42
Grau II (26-50%)	68	21.94
Grau III (51-75%)	46	14.84
Grau IV (76-100%)	11	3.55
Grau V (>100%)	11	3.55

Distribuição do Número de Anormalidades por Registro

Nº de anormalidades	Frequência (registros)	Distribuição
0	10	3.2%
1	47	15.2%
2	75	24.2%
3	72	23.2%
4	84	27.1%
5	22	7.1%

Fonte: Resultado do Análise de Exploratoria (2026) .

Através do trecho de código 8 destaca-se o resultado da figura 8 graficamente com o valor da quantidade de registros para cada variável do dataset, assim demonstrado o número de registros com anormalidade e normalidade, usando a biblioteca `import matplotlib.pyplot as plt`.

Trecho de Código 8

```
fig, axes = plt.subplots(2, 3, figsize=(1920/100, 1080/100))

fig.suptitle('Análise Exploratória da Coluna Vertebral',
             fontsize=20, fontweight='bold', y=0.98)

ax1 = axes[0, 0]
ax2 = axes[0, 1]
ax3 = axes[0, 2]
ax4 = axes[1, 0]
ax5 = axes[1, 1]
ax6 = axes[1, 2]

# GRÁFICO 1: Incidência Pélvica

df_tratados['Classificacao_IP'] = df_tratados['pelvic incidence'].apply(
    lambda ip: 'Normal' if 40 <= ip <= 65 else 'Anormal')

contagem_ip =
df_tratados['Classificacao_IP'].value_counts().reindex(['Anormal',
'Normal'])

cores1 = ['coral', 'lightblue']

ax1.bar(['Anormal', 'Normal'], contagem_ip, color=cores1)

ax1.set_title("Classificação da Incidência Pélvica (IP)\nReferência:
40-65°", fontsize=10)

ax1.set_xlabel("Classificação", fontsize=9)

ax1.set_ylabel("Quantidade de registros", fontsize=9)

ax1.grid(axis='y', alpha=0.3)

ax1.set_ylim(0, max(contagem_ip) + 20)

for i, v in enumerate(contagem_ip):
    ax1.text(i, v + 0.5, f'{v}', ha='center', va='bottom', fontsize=10,
fontweight='bold')

# GRÁFICO 2: Versão Pélvica

df_tratados['Classificacao_VP'] = df_tratados['pelvic tilt'].apply(
    lambda ip: 'Normal' if 10 <= ip <= 25 else 'Anormal')

contagem_ip =
df_tratados['Classificacao_VP'].value_counts().reindex(['Anormal',
'Normal'])

cores2 = ['coral', 'lightblue']
```

```

ax2.bar(['Anormal', 'Normal'], contagem_ip, color=cores2)

ax2.set_title("Classificação da Versão Pélvica (VP)\nReferência:
10-25°", fontsize=10)

ax2.set_xlabel("Classificação", fontsize=9)

ax2.set_ylabel("Quantidade de registros", fontsize=9)

ax2.grid(axis='y', alpha=0.3)

ax2.set_ylim(0, max(contagem_ip) + 20)

for i, v in enumerate(contagem_ip):

    ax2.text(i, v + 0.5, f'{v}', ha='center', va='bottom', fontsize=10,
fontweight='bold')

# GRÁFICO 3: Lordose Lombar

df_tratados['Classificacao_LL'] = df_tratados[' lumbar lordosis
angle'].apply(

    lambda ip: 'Normal' if 40 <= ip <= 80 else 'Anormal')

contagem_ip =
df_tratados['Classificacao_LL'].value_counts().reindex(['Anormal',
'Normal'])

cores3 = ['coral', 'lightblue']

ax3.bar(['Anormal', 'Normal'], contagem_ip, color=cores3)

ax3.set_title("Classificação da Lordose Lombar (LL)\nReferência:
40-80°", fontsize=10)

ax3.set_xlabel("Classificação", fontsize=9)

ax3.set_ylabel("Quantidade de registros", fontsize=9)

ax3.grid(axis='y', alpha=0.3)

ax3.set_ylim(0, max(contagem_ip) + 20)

for i, v in enumerate(contagem_ip):

    ax3.text(i, v + 0.5, f'{v}', ha='center', va='bottom', fontsize=10,
fontweight='bold')

# GRÁFICO 4: Inclinação Sacral

df_tratados['Classificacao_IS'] = df_tratados[' sacral slope'].apply(

    lambda ip: 'Normal' if 30 <= ip <= 50 else 'Anormal')

contagem_ip =
df_tratados['Classificacao_IS'].value_counts().reindex(['Anormal',
'Normal'])

cores4 = ['coral', 'lightblue']

ax4.bar(['Anormal', 'Normal'], contagem_ip, color=cores3)

```

```

ax4.set_title("Classificação da Inclinação Sacral (IS)\nReferência:
30-50°", fontsize=10)

ax4.set_xlabel("Classificação", fontsize=9)

ax4.set_ylabel("Quantidade de registros", fontsize=9)

ax4.grid(axis='y', alpha=0.3)

ax4.set_ylim(0, max(contagem_ip) + 20)

for i, v in enumerate(contagem_ip):

    ax4.text(i, v + 0.5, f'{v}', ha='center', va='bottom', fontsize=10,
fontweight='bold')

# GRÁFICO 5: Raio Pélvico

df_tratados['Classificacao_RP'] = df_tratados['pelvic radius'].apply(

    lambda pr: 'Anormal' if pr < 83.7 else ('Normal' if pr >= 92.9 else
'Indefinido'))

contagem_ip =
df_tratados['Classificacao_RP'].value_counts().reindex(['Anormal',
'Indefinido', 'Normal'])

cores5 = ['#e67e22', '#f1c40f', '#2ecc71']

ax5.bar(['Anormal', 'Indefinido', 'Normal'], contagem_ip, color=cores5)

ax5.set_title("Classificação da Versão Pélvica (VP)\nReferência:
83.7-92.9°", fontsize=10)

ax5.set_xlabel("Classificação", fontsize=9)

ax5.set_ylabel("Quantidade de registros", fontsize=9)

ax5.grid(axis='y', alpha=0.3)

ax5.set_ylim(0, max(contagem_ip) + 20)

for i, v in enumerate(contagem_ip):

    ax5.text(i, v + 0.5, f'{v}', ha='center', va='bottom', fontsize=10,
fontweight='bold')

# GRÁFICO 6: Classificação dos registros

contagem_att =
df_tratados['Class_att'].value_counts().reindex(['Anormal', 'Normal'])

cores6 = ['coral', 'lightblue']

ax6.bar(['Anormal', 'Normal'], contagem_att, color=cores6)

ax6.set_title("Classificação de Registros\n(Coluna Vertebral)",
fontsize=10)

ax6.set_xlabel("Classificação", fontsize=9)

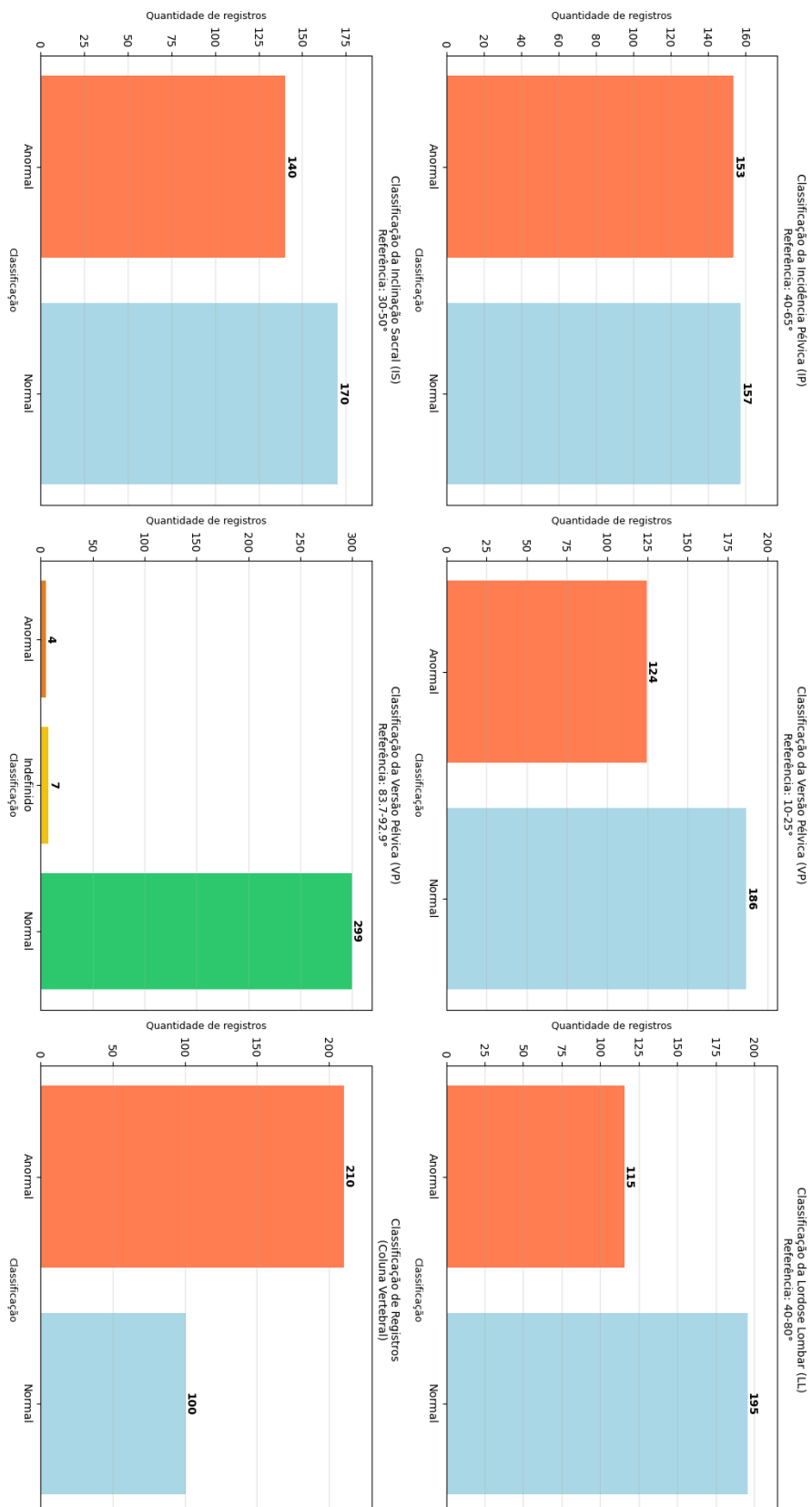
ax6.set_ylabel("Quantidade de registros", fontsize=9)

```

```
ax6.grid(axis='y', alpha=0.3)
ax6.set_ylim(0, max(contagem_att) + 20)
for i, v in enumerate(contagem_att):
    ax6.text(i, v + 0.5, f'{v}', ha='center', va='bottom', fontsize=10,
fontweight='bold')
plt.tight_layout()
plt.show()
```

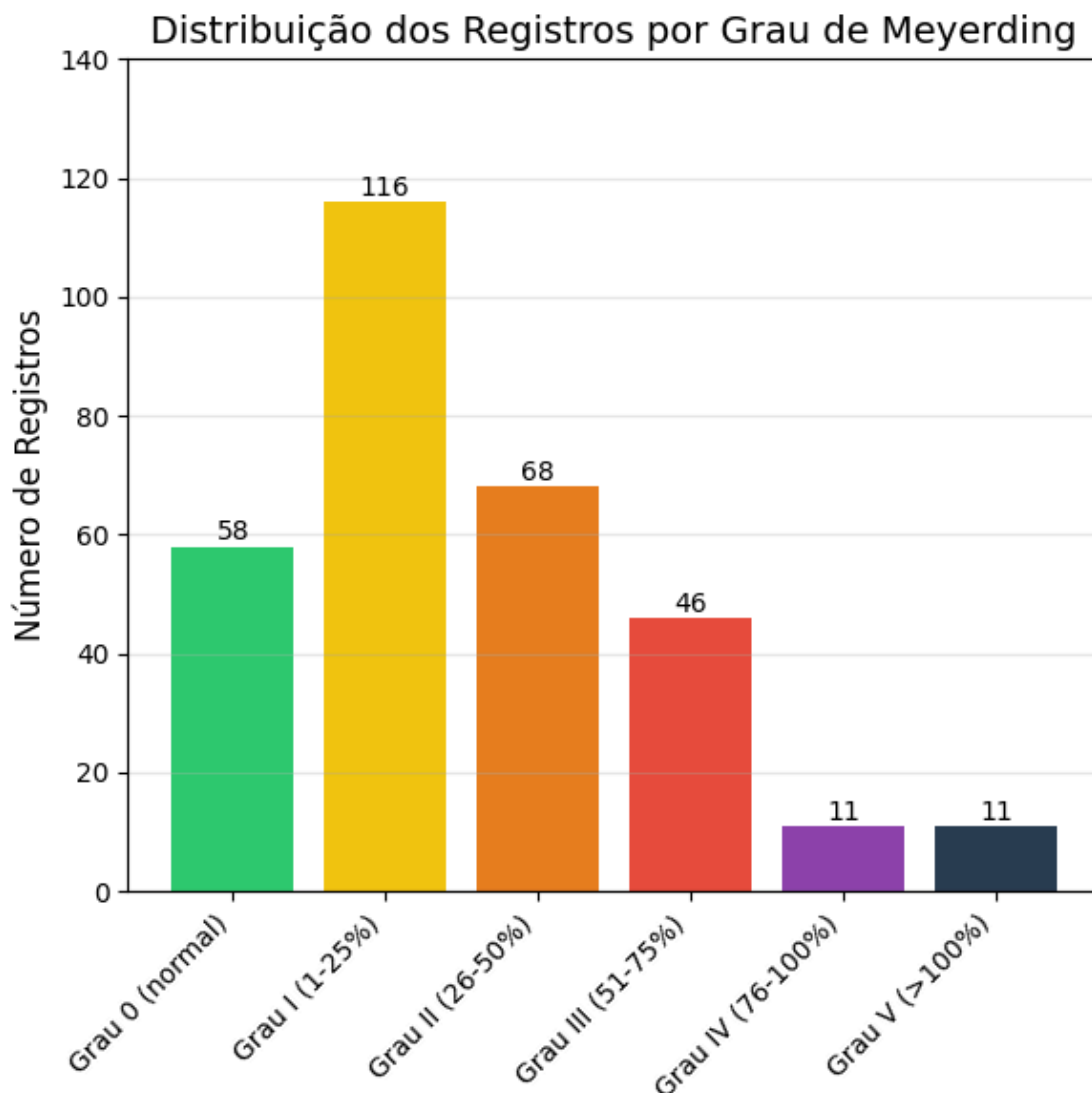
Figura 7 – Gráfico da Análise Exploratória

Análise Exploratória da Coluna Vertebral



Fonte: Resultado do Análise de Exploratoria (2026) .

Figura 8 – Gráfico da Distribuição do Grau de Meyerding



Fonte: Resultado do Análise de Exploratoria (2026) .

4. Resultados e Discussão

4.1. Random Forest

4.1.1. Configuração do Modelo e Divisão dos Dados

O Random Forest Classifier foi configurado com 6 variáveis preditoras para a classificação no conjunto de dados da coluna vertebral. A configuração buscou otimizar o desempenho do modelo e a prevenção do overfitting:

- `n_estimators` (100): Determina o número de 200 árvores de decisão na floresta, conferindo maior robustez e estabilidade à predição final através da agregação de resultados.

- `max_depth (10)`: Limita a profundidade de cada árvore a 10, o que é essencial para prevenir o overfitting ao restringir a complexidade do modelo.
- `min_samples_split (5)`: Define o mínimo de 5 amostras para que um nó interno possa ser dividido, garantindo que as divisões sejam estatisticamente significativas.
- `min_samples_leaf (2)`: Assegura um mínimo de 2 amostras em cada nó folha (terminal), suavizando a decisão e reduzindo a variância.
- `max_features ('sqrt')`: Utiliza a raiz quadrada das features (variáveis) em cada divisão. Esta aleatoriedade é a base do Random Forest, crucial para promover a diversidade das árvores.
- `bootstrap (True)`: Indica que a amostragem dos dados para o treinamento de cada árvore é feita com reposição, um procedimento que garante a diversidade dos subconjuntos de treinamento.
- `random_state (42)`: Fixa a semente aleatória, assegurando a reprodutibilidade completa do modelo e dos resultados.
- `class_weight ('balanced')`: Ajusta automaticamente os pesos das classes. É fundamental para mitigar o viés em datasets desbalanceados, dando maior importância à classe minoritária (anormal).

Abaixo o código completo com a preparação das features, determinando o formato dos dados que serão utilizados para o treinamento e o alvo que é o resultado dos registros, em seguida fazendo divisão dos dados para treinamento e ajustando a normalização dos dados para o funcionamento do modelo, e como destacado acima configurando o modelo e por fim o comando para iniciar o treinamento e as medições necessárias para determinar a eficácia do modelo.

Trecho de Código 9

```
# Preparando dados para o algoritmo RANDOM FOREST

features = ['pelvic incidence', 'pelvic tilt', 'lumbar lordosis angle', 'sacral slope', 'pelvic radius', 'grade of spondylolisthesis']

# Formato dos dados

x = df_tratados[features]
y = df_tratados['Class_att']

# Divisão do conjunto de dados

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.3, random_state=42, stratify=y)

mapeamento = {'Anormal': 1, 'Normal': 0}
```

```

y_numeric = y.map(mapeamento)
y_train_numeric = y_train.map(mapeamento)
y_test_numeric = y_test.map(mapeamento)
# Configurando o algoritmo RANDOM FOREST
model = RandomForestClassifier(
    n_estimators=100,
    max_depth=10,
    min_samples_split=5,
    min_samples_leaf=2,
    max_features='sqrt',
    bootstrap=True,
    random_state=42,
    class_weight='balanced'
)
# Treinamento do modelo
model.fit(x_train, y_train)
# Medições de treinamento
y_pred = model.predict(x_test)
y_pred_proba = model.predict_proba(x_test)[:,1]
accuracy = accuracy_score(y_test, y_pred)
auc_roc = roc_auc_score(y_test, y_pred_proba)
report_dict = classification_report(y_test, y_pred,
target_names=['Anormal', 'Normal'], output_dict=True)

```

4.1.2. Avaliação do Desempenho

O modelo alcançou uma Acurácia de 0.8172 (81,72%) no conjunto de testes, indicando o percentual total de previsões corretas, mesmo que os dados do dataset apresentam um desbalanceamento, indicando maior número de anormalidades. Sendo que, a métrica AUC-ROC foi de 0.9328, um valor muito distante ao classificador aleatório (0.50), sugerindo que o modelo possui uma excelente distinção das classes.

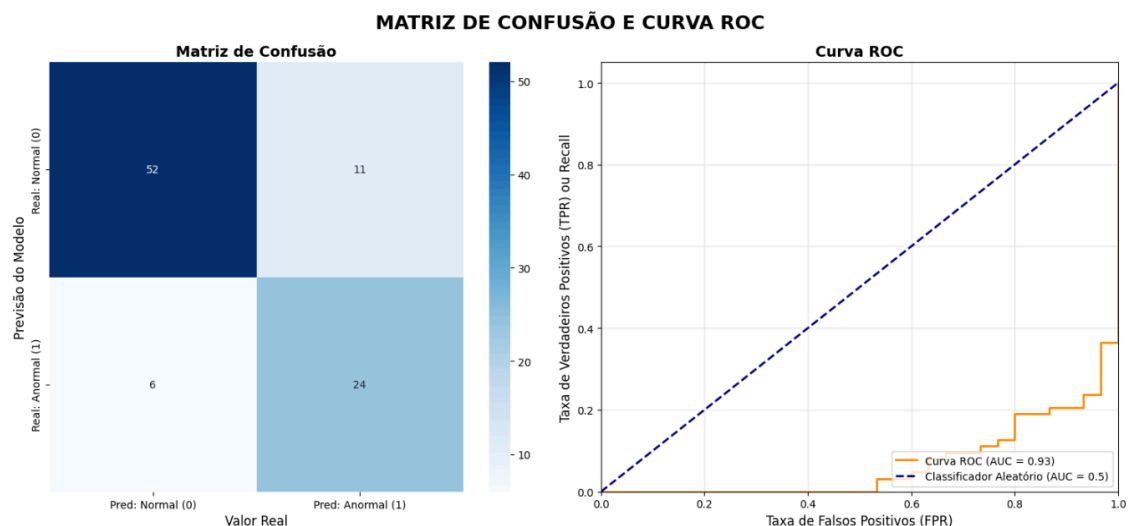
A Matriz de Confusão e o Relatório Detalhado de Classificação (ver Figura 9) confirmam este equilíbrio.

- Verdadeiros Negativos (Normais corretos): 52 registros.

- Verdadeiros Positivos (Anormais corretos): 24 registros.
- Falsos Positivos (Anormais incorretos): 11 registros.
- Falsos Negativos ('Normais incorretos): 6 registros.
- O Recall para anormal apresentou 0,8225 e normal 0,8000.
- A Precisão para anormal 0,897 e normal 0,686.
- O F1-Score para anormal 0,860 e normal 0,738.

Assim, destaca-se que o modelo distinguiu de forma excelente os registros anormais, apenas apresentou uma precisão mais baixa ao distinguir os registros normais que seria a classe minoritária do dataset. Mas no âmbito geral o modelo apresenta um excelente poder de discriminação.

Figura 9 – Gráfico de Matriz de Confusão e Curva ROC



Fonte: Resultado da Aplicação do Modelo Random Forest (2026) .

Trecho de Código 10

```
fig, axes = plt.subplots(1, 2, figsize=(15, 7))
fig.suptitle('MATRIZ DE CONFUSÃO E CURVA ROC',
             fontsize=18, fontweight='bold', y=0.98)
# GRÁFICO 1: Matriz de Confusão
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Pred: Normal (0)', 'Pred: Anormal (1)'],
            yticklabels=['Real: Normal (0)', 'Real: Anormal (1)'],
```

```

ax=axes[0])

axes[0].set_title('Matriz de Confusão', fontsize=16, fontweight='bold')
axes[0].set_xlabel('Valor Real', fontsize=16)
axes[0].set_ylabel('Previsão do Modelo', fontsize=16)

# GRÁFICO 2: Curva ROC

fpr, tpr, thresholds = roc_curve(y_test_numeric, y_pred_proba)

axes[1].plot(fpr, tpr, color='darkorange', lw=2, label=f'Curva ROC (AUC = {auc_roc:.2f})')

axes[1].plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--',
label='Classificador Aleatório (AUC = 0.5)')

axes[1].set_xlim([0.0, 1.0])
axes[1].set_ylim([0.0, 1.05])

axes[1].set_xlabel('Taxa de Falsos Positivos (FPR)', fontsize=12)
axes[1].set_ylabel('Taxa de Verdadeiros Positivos (TPR) ou Recall',
fontsize=12)

axes[1].set_title('Curva ROC', fontsize=14, fontweight='bold')
axes[1].legend(loc="lower right", fontsize=10)
axes[1].grid(True, alpha=0.3)

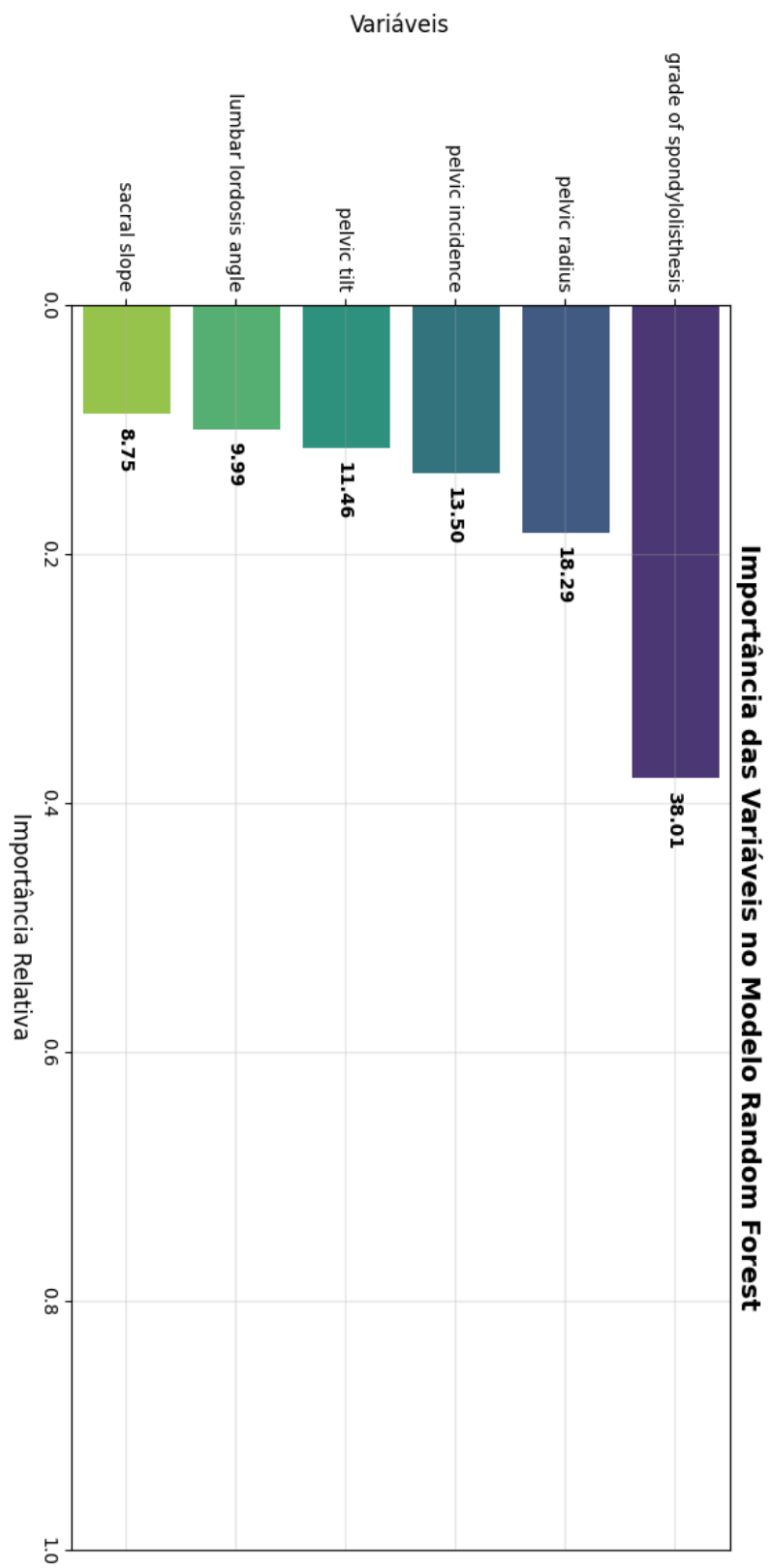
plt.tight_layout()
plt.show()

```

4.1.3. Importância das Variáveis

A análise da importância das variáveis no modelo Random Forest (ver Figura 10) indicou que a variável Grau de Espondilolistese apresentou maior relevância com 38,01% e a variável de inclinação sacral menor relevância 8,75%, assim concluindo que a variável de maior relevância é o que garante um poder preditivo melhor. Sendo o Trecho de Código 11 responsável pela geração do gráfico.

Figura 10 – Gráfico de Importância das Variáveis do Random Forest



Fonte: Resultado da Importância das Variáveis (2026) .

Trecho de Código 11:

```

# Analisar o nível de importancia dos fatores

importances = model.feature_importances_

feature_names = x.columns

importance_df = pd.DataFrame({'feature': feature_names,
                              'importance': importances}).sort_values('importance', ascending=False)

plt.figure(figsize=(12,6))

sns.barplot(data=importance_df, x='importance', y='feature',
            hue='feature', palette='viridis', legend=False)

plt.title('Importância das Variáveis no Modelo Random Forest',
          fontsize=14, fontweight='bold')

plt.xlabel('Importância Relativa', fontsize=12)

plt.ylabel('Variáveis', fontsize=12)

plt.xlim(0, 1)

plt.grid(True, alpha=0.3)

for i, v in enumerate(importance_df['importance']):
    plt.text(v + 0.01, i, f'{v*100:.2f}', va='center', fontweight='bold')

plt.tight_layout()

plt.show()

```

4.2 K-Means

4.2.1. Aplicando o Modelo K-Means

Aplicando as bibliotecas descritas no tópico acima (Trecho de Código 1), o trecho de código 12 é um método para determinar o número ideal de clusters e juntamente com a geração do gráfico da Figura 10.

Trecho de Código 12

```

scaler = StandardScaler()

x_scaled = scaler.fit_transform(x_train)

wcss = []

for i in range(1,11):

    kmeans = KMeans(n_clusters=i, init='k-means++', max_iter=300,
                    n_init=10, random_state=42)

    kmeans.fit(x_scaled)

```

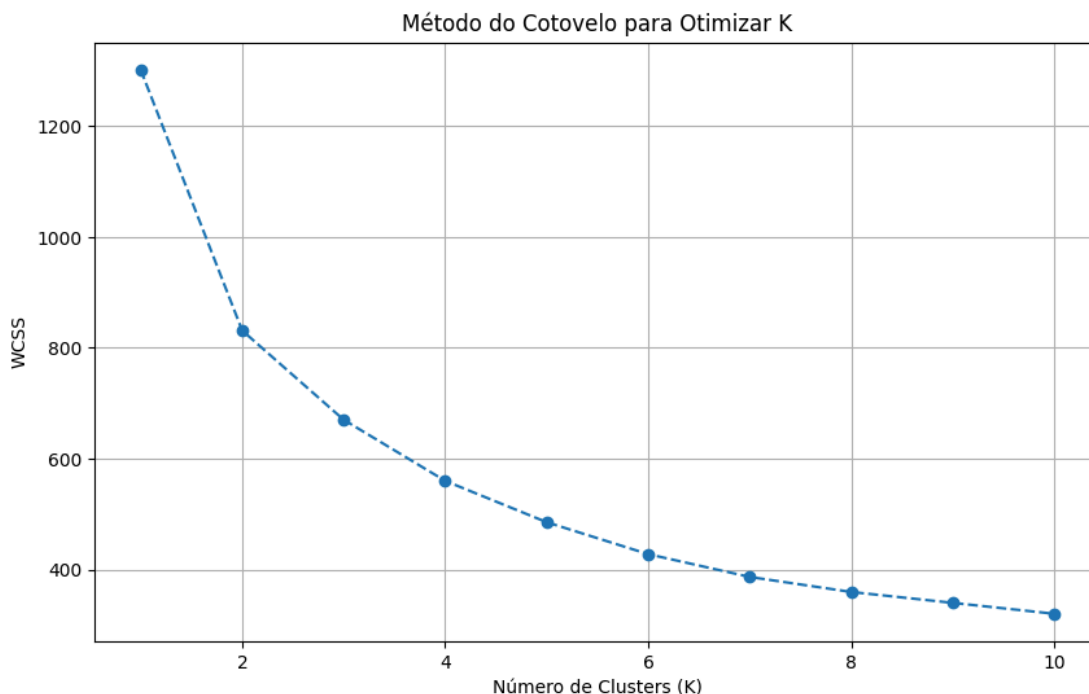


```

wcss.append(kmeans.inertia_)
plt.figure(figsize=(10, 6))
plt.plot(range(1, 11), wcss, marker='o', linestyle='--')
plt.title('Método do Cotovelo para Otimizar K')
plt.xlabel('Número de Clusters (K)')
plt.ylabel('WCSS')
plt.grid(True)
plt.show()

```

Figura 10 – Gráfico do Método do Cotovelo para Indicar o K Ideal



Fonte: Resultado do Modelo K-Means (2026) .

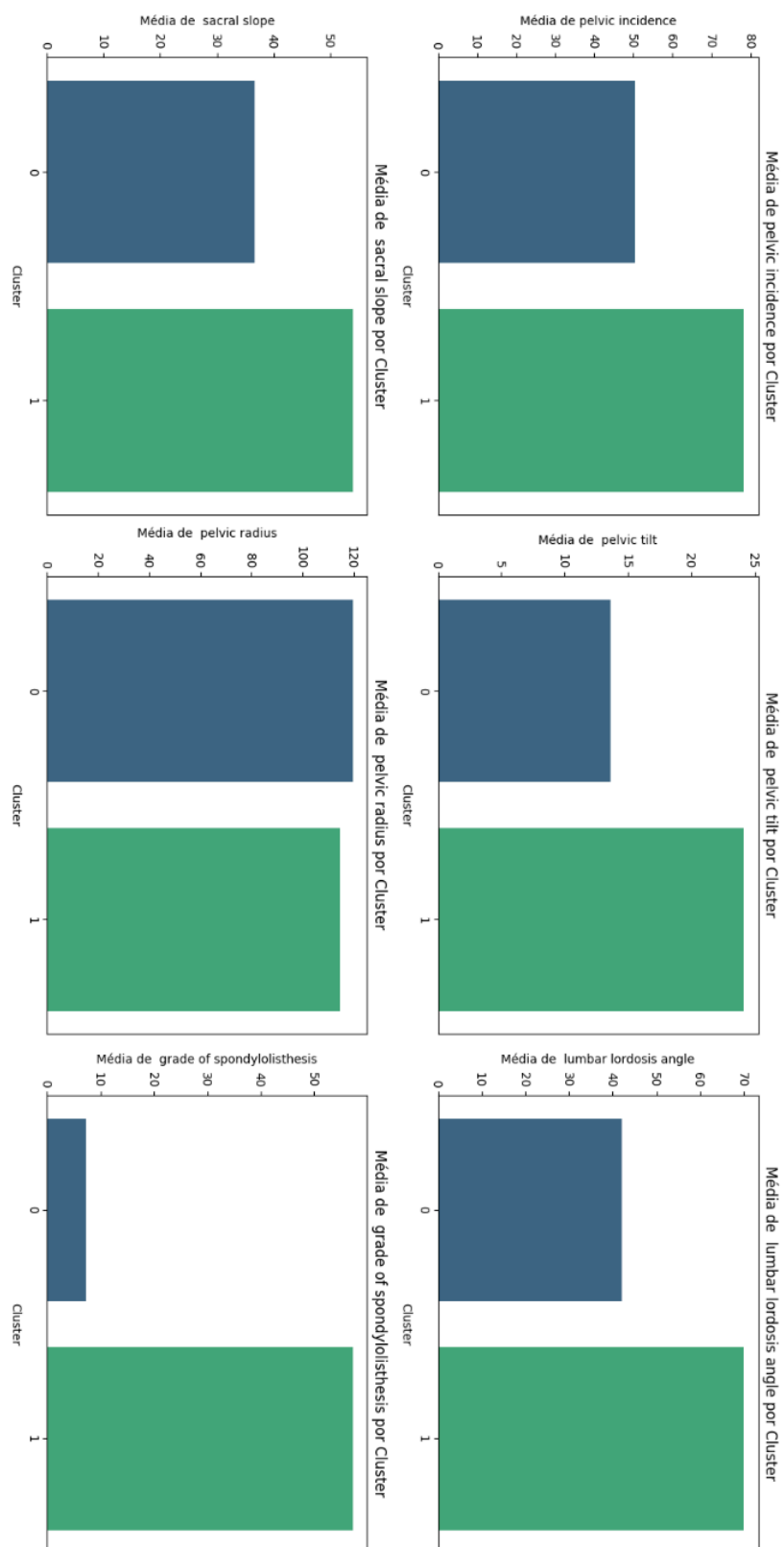
4.2.2. Agrupamento e Avaliação Externa

Aplicou-se o algoritmo K-Means aos dados escalonados, utilizando o Método do Cotovelo para determinar o número ideal de clusters, demonstrado na Figura 10. O valor selecionado foi K=2, sendo aplicado o código para determinar os grupos dos clusters demonstrados no Trecho de Código 13.

Através da escolha do k ideal, gera-se um gráfico demonstrando o perfil médio dos clusters, assim revelado na Figura 11.

Figura 11 – Gráfico de Identificação de Perfil Médio

Perfil Médio das Características por Cluster (K=2)



Fonte: Resultado do Modelo K-Means (2026) .

```

K_IDEAL = 2

kmeans_final = KMeans(n_clusters=K_IDEAL, init='k-means++',
max_iter=300, n_init=10, random_state=42)

cluster_labels = kmeans_final.fit_predict(x_scaled)

x_train_df = pd.DataFrame(x_train, columns=x_train.columns)

x_train_df['Cluster'] = cluster_labels

```

Trecho de Código 14

```

df_plot = analyse_clusters.reset_index()

features = [col for col in df_plot.columns if col != 'Cluster']

num_features = len(features)

n_cols = 3

n_rows = (num_features + n_cols - 1) // n_cols

plt.figure(figsize=(18, 5 * n_rows))

plt.suptitle(f'Perfil Médio das Características por Cluster
(K={K_IDEAL})', fontsize=16, y=1.02)

for i, feature in enumerate(features):

    plt.subplot(n_rows, n_cols, i + 1)

    sns.barplot(x='Cluster', y=feature, hue='Cluster', data=df_plot,
palette='viridis', legend=False)

    plt.title(f'Média de {feature} por Cluster', fontsize=12)

    plt.ylabel(f'Média de {feature}')

    plt.xlabel('Cluster')

plt.tight_layout(rect=[0, 0.03, 1, 0.98])

plt.savefig('perfis_clusters_k_means.png')

plt.show()

```

Por fim através do Trecho de Código 14 gerou-se a avaliação externa, comparando os clusters descobertos com a variável alvo real (*Class_att*) através de métricas de concordância, resultou em um Adjusted Rand Index (ARI) de 0.0995 e Normalized Mutual Information (NMI) de 0.1600. Estes valores próximos de zero demonstram que o agrupamento não conseguiu segregar os registros em grupos que se alinhassem com o status real de anormalidades na coluna vertebral, demonstrado na imagem da Figura 11.

Figura 11 – Resultado de Avaliação Externa

Avaliação Externa do K-Means (K=2)

✓ Adjusted Rand Index (ARI)

0.0995

⚠ Normalized Mutual Information (NMI)

0.1600

Matriz de Contingência (Real vs Cluster)

Cluster	0	1
Real		
Anormal	66	81
Normal	63	7

Fonte: Resultado do Modelo K-Means (2026) .

A matriz de contingência apresentou que o cluster 0 misturou-se aos resultados assim não determinando o isolamento e o cluster 1 isolou de forma mais eficiente os registros, inclinando mais para a anormalidade.

Aplicando um número maior de clusters o resultado apresentou mais uma melhora no NMI, mais ainda abaixo do valor ideal de 1, e ARI apresentou piora, indicando aleatoriedade dos dados.

Figura 12 – Resultado de Avaliação Externa

Avaliação Externa do K-Means (K=8)

✓ Adjusted Rand Index (ARI)

0.0669

⚠ Normalized Mutual Information (NMI)

0.1781

Matriz de Contingência (Real vs Cluster)

Cluster	0	1	2	3	4	5	6	7
Real								
Anormal	9	4	19	29	28	33	24	1
Normal	0	32	2	20	2	14	0	0

Fonte: Resultado do Modelo K-Means (2026) .

5. Conclusão

Este estudo avaliou a aplicação dos algoritmos Random Forest e K-Means para previsão e agrupamento de um dataset de registros referente à coluna vertebral. O modelo Random Forest alcançou uma acurácia geral de 81,72%, mas apresentou na precisão de classificar o Normal um valor mais baixo de 0,686, assim gerando falsos positivos, mais com um Recall de 0.825 para anormal e 0,800 para normal, o modelo apresenta uma sensibilidade satisfatória, provavelmente, pelo desbalanceamento do dataset 209 anormais e 100 normais.

A análise de importância das variáveis forneceu um insight clínico crucial, destacando o Grau de Espondilolistese como o fator mais preditivo. Por outro lado, o K-Means demonstrou ser ineficaz neste contexto, mesmo aumentando o número de clusters, não conseguindo gerar correlação com a situação clínica real dos registros.

Como trabalho futuro, é essencial a aplicação de técnicas de balanceamento de classes no dataset para mitigar o viés do modelo Random Forest em favor da classe minoritárias, visando uma melhora na precisão.

6. Referências

- FLORES, Fernando. *Conheça os tipos e fatores de risco das alterações posturais*. 21 jul. 2025. Disponível em: <https://www.drfernandoflores.com.br/tipos-e-fatores-de-riscos-das-alteracoes-posturais>. Acesso em: 6 jun. 2026.
- EKŞİ, M. Şakir et al. **Is diabetes mellitus a risk factor for modic changes?: A novel model to understand the association between intervertebral disc degeneration and end-plate changes**. *Journal of Orthopaedic Science*, v. 25, n. 4, p. 571-575, jul. 2020. DOI 10.1016/j.jos.2019.09.005. Disponível em: <https://pubmed.ncbi.nlm.nih.gov/31564384/>. Acesso em: 6 jun. 2026.
- RAHME, R.; MOUSSA, R. **Modic changes: their pathophysiological and clinical significance**. *Neurochirurgie*, v. 54, n. 1, p. 1-8, fev. 2008. DOI 10.1016/j.neuchi.2007.10.002. Disponível em: <https://pubmed.ncbi.nlm.nih.gov/18194532/>. Acesso em: 6 jun. 2026.
- MELEPURA, Febin. **What causes lower back pain in females? A NYC pain specialist explains**. *SPIN NYC Blog, New York*, 3 jun. 2026. Disponível em: <https://www.sportspainmanagementnyc.com/blog/what-are-common-causes-of-lower-back-pain-in-women>. Acesso em: 6 jun. 2026.
- TEIXEIRA, Ricardo. **Lordose lombar: o que realmente importa? Dr. Ricardo Teixeira - Ortopedia e Cirurgia da Coluna, São Paulo**, 2 ago. 2024. Disponível em: <https://dr ricardoteixeira.com.br/lordose-lombar/>. Acesso em: 6 jun. 2026.
- RADIOLOGIA ONLINE. **Medidas ortopédicas**. [S.l.], 26 dez. 2017. Disponível em: <https://radiologia.online/medidas-ortopedicas/>. Acesso em: 6 jun. 2026.
- SERGIDES, I. G. et al. **Lumbo-pelvic lordosis and the pelvic radius technique in the assessment of spinal sagittal balance: strengths and caveats**. *European Spine Journal*, [s.l.], v. 20, n. Suppl 5, p. 591-601, 24 ago. 2011. DOI 10.1007/s00586-011-1926-z. Disponível em: <https://pmc.ncbi.nlm.nih.gov/articles/PMC3175929/>. Acesso em: 6 jun. 2026.
- RODRIGUES, Thiago. **Espondilolistese: tudo o que você precisa saber!**. *Neurocirurgia | Dr. Thiago Rodrigues MD, PhD*, 2022. Disponível em: <https://neurocirurgiasp.com.br/artigos/servico-de-neurocirurgia/o-que-e-espondilolistese/>. Acesso em: 6 jun. 2026.
- JSR90. **Vertebral Column Data Set**. Kaggle, 21 jul. 2022. Disponível em: <https://www.kaggle.com/datasets/jessanrod3/vertebralcolumndataset/data>. Acesso em: 1 jun 2026.